

Prime

素数

素数与合数的定义，见 [数论基础](#)。

素数计数函数：小于或等于 x 的素数的个数，用 $\pi(x)$ 表示。随着 x 的增大，有这样的近似结果： $\pi(x) \sim \frac{x}{\ln x}$ 。

素性测试

素性测试（Primality test）可以用于判定所给自然数是否为素数。

素性测试有两种：

1. 确定性测试：绝对确定一个数是否为素数。常见例子包括试除法、Lucas–Lehmer 测试和椭圆曲线素性证明。
2. 概率性测试：通常比确定性测试快很多，但有可能（尽管概率很小）错误地将 [合数](#) 识别为质数（尽管反之则不会）。因此，通过概率素性测试的数字被称为 **可能素数**，直到它们的素数可以被确定性地证明。而通过测试但实际上是合数的数字则被称为 **伪素数**。有许多特定类型的伪素数，最常见的是费马伪素数，它们是满足费马小定理的合数。概率性测试的常见例子包括 Miller–Rabin 测试。

试除法

暴力做法自然可以枚举从小到大的每个数看是否能整除。

▼ 参考实现



C++Python

```
1 bool isPrime(int a) {
2     if (a < 2) return false;
3     for (int i = 2; i < a; ++i)
4         if (a % i == 0) return false;
5     return true;
6 }
```

```
1 def isPrime(a):
2     if a < 2:
3         return False
4     for i in range(2, a):
5         if a % i == 0:
6             return False
7     return True
```

这样做是十分稳妥了，但是真的有必要每个数都去判断吗？

很容易发现这样一个事实：如果 a 是 b 的约数，那么 b 也是 a 的约数。

这个结论告诉我们，对于每一对 (a, b) ，只需要检验其中的一个就好了。为了方便起见，我们只考察每一对里面小的那个数。不难发现，所有这些较小数就是 $[2, \sqrt{a}]$ 这个区间里的数。

由于 a 肯定是约数，所以不检验它。

▼ 参考实现



C++Python

```

1 bool isPrime(int a) {
2     if (a < 2) return 0;
3     for (int i = 2; (long long)i * i <= a; ++i)    // 防溢出
4         if (a % i == 0) return 0;
5     return 1;
6 }

```

```

1 def isPrime(a):
2     if a < 2:
3         return False
4     for i in range(2, int(sqrt(a)) + 1):
5         if a % i == 0:
6             return False
7     return True

```

Fermat 素性测试

Fermat 素性检验 是最简单的概率性素性检验。

我们可以根据 [费马小定理](#) 得出一种检验素数的思路：

基本思想是不断地选取在 a 中的基 b ，并检验是否每次都有 $b^{a-1} \equiv 1 \pmod{a}$ 。

▼ 参考实现



C++Python

```

1 bool millerRabin(int n) {
2     if (n < 3) return n == 2;
3     // test_time 为测试次数, 建议设为不小于 8
4     // 的整数以保证正确率, 但也不宜过大, 否则会影响效率
5     for (int i = 1; i <= test_time; ++i) {
6         int a = rand() % (n - 2) + 2;
7         if (quickPow(a, n - 1, n) != 1) return false;
8     }
9     return true;
10 }

```

```

1 def millerRabin(n):
2     if n < 3:
3         return n == 2
4     # test_time 为测试次数, 建议设为不小于 8
5     # 的整数以保证正确率, 但也不宜过大, 否则会影响效率
6     for i in range(1, test_time + 1):
7         a = random.randint(0, 32767) % (n - 2) + 2
8         if quickPow(a, n - 1, n) != 1:
9             return False
10     return True

```

如果 a 但不是素数，则 a 被称为以 a 为底的 **伪素数**。我们在实践中观察到，如果 a ，那么 a 通常是素数。但这里也有个反例：如果 a 且 a ，即使 a 是合数，有 a 。事实上， a 是最小的伪素数基数。

很遗憾，费马小定理的逆定理并不成立，换言之，满足了 $a^{b-1} \equiv 1 \pmod{a}$ ，也不一定是素数。甚至有些合数 a 满足对任意满足 a 的整数 b 均有 $a^{b-1} \equiv 1 \pmod{a}$ ，这样的数称为 [Carmichael 数](#)。

Carmichael 函数

对正整数 a ，定义 Carmichael 函数（卡迈克尔函数）为对任意满足 a 的整数 b ，使

恒成立的最小正整数。

即：

Carmichael 函数有如下性质：

1. (**Carmichael 定理**) 对任意素数 p 和任意正整数 a ，

► 证明

进而有：

1. 对任意正整数 a ，有
 2. 对任意正整数 a ，有
2. 令 n 的唯一分解式为 $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ ，则

由 [中国剩余定理](#) 和 Carmichael 定理易证。

进而有：

1. 对任意正整数 a ，有

Carmichael 数

对于合数 n ，如果对于所有正整数 a ， a 和 n 互素，都有同余式 $a^n \equiv a \pmod{n}$ 成立，则合数 n 为 **Carmichael 数**（卡迈克尔数，[OEIS:A002997](#)）。

比如 561 就是一个 Carmichael 数，同时也是最小的 Carmichael 数。

我们可以用如下方法判断合数 n 是否为 Carmichael 数：

▼ Korselt 判别法¹

合数 n 是 Carmichael 数当且仅当 n 无平方因子且对 n 的任意质因子 p 均有 $p-1 \mid n-1$ 。

上述判别法可简化为：

▼ Carmichael 数判别法

合数 n 是 Carmichael 数当且仅当，其中 $\lambda(n)$ 为 [Carmichael 函数](#)。

Carmichael 数有如下性质：

1. Carmichael 数无平方因子且至少有 3 个不同的质因子。
2. 设 $\pi(x)$ 为小于 x 的 Carmichael 数个数，则：

1. (Alford, Granville, Pomerance. 1994²) 。

由此可知 Carmichael 数有无限多个。

2. (Erdős. 1956³)，其中 c 为常数。

由此可知 Carmichael 数的分布十分稀疏。实际上，[4](#)。

▼ 注意

「若 n 为 Carmichael 数，则 $a^{n-1} \equiv 1 \pmod n$ 也为 Carmichael 数」是错误的。

如 $n=561$ 为 Carmichael 数，考虑 $a=2$ 。

注意到，由 Korselt 判别法知，若 n 是 Carmichael 数，则 $n-1$ 和 $a-1$ 均为 n 的因子。

而 $2^{560} \not\equiv 1 \pmod{561}$ ，故 561 因此不是 Carmichael 数。

Miller–Rabin 素性测试

Miller–Rabin 素性测试 (Miller–Rabin primality test) 是更好的素数判定方法。它是由 Miller 和 Rabin 二人根据 Fermat 素性测试优化得到的。和其它概率性素数测试一样，它也只能检测出伪素数。要确保是素数，需要用慢得多的确定性算法。然而，实际上没有已知的数字通过了 Miller–Rabin 测试等高级概率性测试但实际上却是合数，因此我们可以放心使用。

在不考虑乘法的复杂度时，对数 n 进行 k 轮测试的时间复杂度是 $O(k \log^3 n)$ 。Miller-Rabbin 素性测试常用于对高精度数进行测试，此时时间复杂度是 $O(k \log^3 n)$ ，利用 FFT 等技术可以优化到 $O(k \log^2 n)$ 。

二次探测定理

如果 n 是奇素数，则 $x^2 \equiv a \pmod n$ 的解为 $x \equiv \pm \sqrt{a} \pmod n$ 或者 $x \equiv \pm \sqrt{-a} \pmod n$ 。

要证明该定理，只需将上面的方程移项，再使用平方差公式，得到 $x^2 - a \equiv 0 \pmod n$ ，即可得出上面的结论。

实现

根据 Carmichael 数的性质，可知其一定不是素数。

不妨将费马小定理和二次探测定理结合起来使用：

将 $n-1$ 中的指数 $n-1$ 分解为 $2^s \cdot d$ ，在每轮测试中对随机出来的 a 先求出 $a^d \pmod n$ ，之后对这个值执行最多 s 次平方操作，若发现非平凡平方根时即可判断出其不是素数，否则再使用 Fermat 素性测试判断。

还有一些实现上的小细节：

- 对于一轮测试，如果某一时刻 $x^2 \equiv a \pmod n$ ，则之后的平方操作全都会得到 x ，则可以直接通过本轮测试。
- 如果找出了一个非平凡平方根 x ，则之后的平方操作全都会得到 x 。可以选择直接返回 `false`，也可以放到 s 次平方操作后再返回 `false`。

这样得到了较正确的 Miller Rabin：（来自 fjzzq2002）

▼ 参考实现



C++Python

```

1 bool millerRabin(int n) {
2     if (n < 3 || n % 2 == 0) return n == 2;
3     if (n % 3 == 0) return n == 3;
4     int u = n - 1, t = 0;
5     while (u % 2 == 0) u /= 2, ++t;
6     // test_time 为测试次数, 建议设为不小于 8
7     // 的整数以保证正确率, 但也不宜过大, 否则会影响效率
8     for (int i = 0; i < test_time; ++i) {
9         // 0, 1, n-1 可以直接通过测试, a 取值范围 [2, n-2]
10        int a = rand() % (n - 3) + 2, v = quickPow(a, u, n);
11        if (v == 1) continue;
12        int s;
13        for (s = 0; s < t; ++s) {
14            if (v == n - 1) break; // 得到平凡平方根 n-1, 通过此轮测试
15            v = (long long)v * v % n;
16        }
17        // 如果找到了非平凡平方根, 则会由于无法提前 break; 而运行到 s == t
18        // 如果 Fermat 素性测试无法通过, 则一直运行到 s == t 前 v 都不会等于 -1
19        if (s == t) return 0;
20    }
21    return 1;
22 }

1 def millerRabin(n):
2     if n < 3 or n % 2 == 0:
3         return n == 2
4     if n % 3 == 0:
5         return n == 3
6     u, t = n - 1, 0
7     while u % 2 == 0:
8         u = u // 2
9         t = t + 1
10    # test_time 为测试次数, 建议设为不小于 8
11    # 的整数以保证正确率, 但也不宜过大, 否则会影响效率
12    for i in range(test_time):
13        # 0, 1, n-1 可以直接通过测试, a 取值范围 [2, n-2]
14        a = random.randint(2, n - 2)
15        v = pow(a, u, n)
16        if v == 1:
17            continue
18        s = 0
19        while s < t:
20            if v == n - 1:
21                break
22            v = v * v % n
23            s = s + 1
24        # 如果找到了非平凡平方根, 则会由于无法提前 break; 而运行到 s == t
25        # 如果 Fermat 素性测试无法通过, 则一直运行到 s == t 前 v 都不会等于 -1
26        if s == t:
27            return False
28    return True

```

另外, 假设 [广义 Riemann 猜想](#) (generalized Riemann hypothesis, GRH) 成立, 则对数 $\pi(x)$ 最多只需要测试 $\sqrt{x}$ 中的全部整数即可确定 $\pi(x)$ 的素性, 证明参见注释 7。

而在 $O(\sqrt{x})$ 范围内, 通常都是对 $\sqrt{x}$ 范围内的数进行素性检验。对于 $\sqrt{x}$ 范围内的数, 选取三个数作为基底进行 Miller–Rabin 素性检验就可以确定素性; 对于 x 范围内的数, 选取七个数作为基底进行 Miller–Rabin 素性检验就可以确定素性。参见注释 8。

也可以选取 $\pi(\sqrt{x})$ (即前 $\pi(\sqrt{x})$ 个素数) 检验 x 范围内的素数。

注意如果要使用上面的数列中的数 作为基底判断 的素性：

- 所有的数都要取一遍，不能只选小于 的；
- 把 换成；
- 如果 或 ，则直接通过该轮测试。

反素数

顾名思义，素数就是因子只有两个的数，那么反素数，就是因子最多的数（并且因子个数相同的时候值最小），所以反素数是相对于一个集合来说的。

一种符合直觉的反素数定义是：在一个正整数集合中，因子最多并且值最小的数，就是反素数。

▼ 反素数

对于某个正整数，如果任何小于 的正数的约数个数都小于 的约数个数，则称为是 **反素数** (anti-prime, a.k.a., highly compositive numbers) 。

▼ 注意

注意区分 [emirp](#)，它表示的是逐位反转后是不同素数的素数（如 149 和 941 均为 emirp，101 不是 emirp）。

过程

那么，如何来求解反素数呢？

首先，既然要求因子数，首先要做的就是素因子分解。把 分解成 的形式，其中 是素数， 为他的指数。这样的话总因子个数就是 。

但是显然质因子分解的复杂度是很高的，并且前一个数的结果不能被后面利用。所以要换个方法。

我们来观察一下反素数的特点。

1. 反素数肯定是从 开始的连续素数的幂次形式的乘积。
2. 数值小的素数的幂次大于等于数值大的素数，即 中，有 。

解释：

1. 如果不是从 开始的连续素数，那么如果幂次不变，把素数变成数值更小的素数，那么此时因子个数不变，但是 的数值变小了。交换到从 开始的连续素数的时候 值最小。
2. 如果数值小的素数的幂次小于数值大的素数的幂，那么如果把这两个素数交换位置（幂次不变），那么所得的因子数量不变，但是 的值变小。

另外还有两个问题，

1. 对于给定的，要枚举到哪一个素数呢？

最极端的情况大不了就是，所以只要连续素数连乘到刚好小于等于 就可以的呢。再大了，连全都一次幂，都用不了，当然就是用不到的啦！

2. 我们要枚举到多少次幂呢？

我们考虑一个极端情况，当我们最小的素数的某个幂次已经比所给的（的最大值）大的话，那么展开成其他的形式，最大幂次一定小于这个幂次。unsigned long long 的最大值是，所以可以枚举到。

细节有了，那么我们具体如何具体实现呢？

我们可以把当前走到每一个素数前面的时候列举成一棵树的根节点，然后一层层的去找。找到什么时候停止呢？

1. 当前走到的数字已经大于我们想要的数字了；
2. 当前枚举的因子已经用不到了；
3. 当前因子大于我们想要的因子了；
4. 当前因子正好是我们想要的因子（此时判断是否需要更新最小）。

然后 dfs 里面不断一层一层枚举次数继续往下迭代可以。

例题

▼ [Codeforces 27E. A number with a given number of divisors](#)

求具有给定除数个数的最小自然数。答案保证不超过。

- 解题思路
- 参考代码

▼ [ZOJ - More Divisors](#)

求不超过 的数中，除数最多的数。

- 解题思路
- 参考代码

参考资料与注释

1. Rui-Juan Jing, Marc Moreno-Maza, Delaram Talaashrafi, "[Complexity Estimates for Fourier-Motzkin Elimination](#)", Journal of Functional Programming 16:2 (2006) pp 197-217.
2. [数论部分第一节：素数与素性测试](#)
3. [Miller-Rabin 与 Pollard-Rho 学习笔记 - Bill Yang's Blog](#)
4. [Primality test - Wikipedia](#)
5. [桃子的算法笔记——反素数详解 \(acm/OI\)](#)
6. [The Rabin-Miller Primality Test](#)
7. Bach, Eric, "[Explicit bounds for primality testing and related problems](#)", Mathematics of Computation, 55:191 (1990) pp 355–380.
8. [Deterministic variant of the Miller-Rabin primality test](#)
9. [Fermat pseudoprime - Wikipedia](#)
10. [Carmichael number - Wikipedia](#)
11. [Carmichael function - Wikipedia](#)
12. [Carmichael Number -- from Wolfram MathWorld](#)
13. [Carmichael's Lambda Function | Brilliant Math & Science Wiki](#)
14. [Highly composite number - Wikipedia](#)

-
1. Korselt, A. R. (1899). "Problème chinois" *L'Intermédiaire des Mathématiciens*. **6**: 142–143. [↩](#)
 2. W. R. Alford; Andrew Granville; Carl Pomerance (1994). "There are Infinitely Many Carmichael Numbers". *Annals of Mathematics*. 140 (3): 703–722. [↩](#)
 3. Erdős, P. (1956). "On pseudoprimes and Carmichael numbers" *Publ. Math. Debrecen*. 4 (3–4): 201–206. [↩](#)
 4. PINCH, Richard GE. The Carmichael numbers up to . [↩](#)
-

本页面最近更新：2024/12/9 22:20:28, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [Marcythm](#), [StudyingFather](#), [383494](#), [abc1763613206](#), [Alpacabla](#), [Back1ght](#), [c-forrest](#), [CCXXI](#), [drkelo](#), [Early0v0](#), [Enter-tainer](#), [Great-designer](#), [greyqz](#), [GuanghaoYe](#), [H-J-Granger](#), [HeRaNO](#), [Ir1d](#), [isdanni](#), [kenlig](#), [ksyx](#), [lazyasn](#), [MegaOwler](#), [Menci](#), [ouuan](#), [shawllew](#), [shopee-jin](#), [shuzhouliu](#), [Siger Young](#), [Tiphereth-A](#), [TrisolarisHD](#), [untitledunrevised](#), [void-mian](#), [Voileexperiments](#), [weilycoder](#), [Xeonacid](#), [xiti1soft](#), [YuzhenQin1](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用